60

BÖLÜM: MAANG EŞZAMANLILIK MÜLAKATI & SISTEM TASARIMI USTALIĞI | MODULE: MAANG CONCURRENCY WHITEBOARD & SYSTEM DESIGN MASTERY

Staff+ Concurrency Mimarı: Canlı Sistem Çöküş Otopsisi & Çok Çekirdekli Ayarlar

The Staff+ Concurrency Architect: Production Post-Mortems & Tuning

⚡ Bellek & Eşzamanlılık Akış Şeması

⚡ Concurrency & Memory Schematic




```
CANLI SİSTEM ÇÖKÜŞÜ (INCIDENT):
- Semptom: 99.9th Percentile Gecikme 50ms'den 12 Saniyeye Fırladı!
- CPU Kullanımı: %98 (128 Çekirdekli Sunucu)

ADLİ PROTOKOL (POST-MORTEM WORKFLOW):
1. METRİKLERİ İNCELE  ──► JVM GC Pauses? (ZGC / Shenandoah kontrolü)
2. THREAD DUMP AL     ──► jcmd <PID> Thread.print ──► Kilit çekişmesi var mı?
3. PROFİLER BAĞLA     ──► async-profiler ──► Cache Misses & Lock Contention
4. MİMARİ MÜDAHALE:
   - Kaba kilitleri Striped Lock veya Lock-Free'e çevir
   - Platform thread havuzlarını Project Loom Sanal Thread'lerine göç ettir
   - False Sharing olan kritik alanlara @Contended dolgusu uygula!
```

💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Bir çırak, araba bozulunca 'Motoru değiştirelim' der. Kıdemli bir usta ise kulağını motora dayar, 3. silindirin contasındaki 1 milimlik yırtığı saniyesinde bulur. Staff+ Concurrency Mimarı, binlerce sunucunun saniyede milyonlarca işlem yaptığı bir MAANG şirketinde sistem kilitlendiğinde paniklemez. 3 dakikada thread dump alır, hangi thread'in hangi kilit yüzünden tıkandığını teşhis eder ve sistemi sıfır veri kaybıyla ayağa kaldırır!

🇬🇧 ENGLISH (ELI5)

A novice engineer sees an outage and suggests restarting all servers. A Staff+ Concurrency Architect approaches a production failure like a master forensic detective. With a 128-core server melting down at 99% CPU and p99 latency spiking to 10 seconds, the Staff Architect captures continuous low-overhead async-profiler flame graphs, identifies a single contested cache line or lock convoy in under 3 minutes, and executes a targeted hot-patch.

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Staff+ Seviyesinde Üretim Optimizasyonu ve Donanım Uyumu (Mechanical Sympathy): 1) async-profiler ile Saftepoint Bias Önleme: Standart Java profiler'ları thread'leri yalnızca Safepoint anlarında örnekler; bu da yanıltıcı sonuçlar verir. `async-profiler` Linux çekirdeğinin `perf_events` alt yapısını kullanarak safepoint bağımsız gerçek CPU döngülerini ve kilit bekleme alev grafiklerini (Flame Graph) çizer. 2) NUMA (Non-Uniform Memory Access) Duyarlılığı: Çok soketli sunucularda bir CPU çekirdeğinin diğer soketin RAM'ine erişmesi yüksek gecikmeye yol açar. JVM `-XX:+UseNUMA` bayrağı ile donatılmalıdır. 3) Modern Çöp Toplayıcılar (ZGC & Shenandoah): Yüksek eşzamanlı sistemlerde STW (Stop-The-World) duraklamalarını 1 milisaniyenin altına indirmek için Generational ZGC (`-XX:+UseZGC`) tercih edilmelidir.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Staff+ Architectural Mastery & Mechanical Sympathy: 1) Safepoint-Bias-Free Profiling: Standard JVM profiler samples exclusively at safepoint intervals, introducing severe observational skew. Deploy `async-profiler` leveraging Linux kernel `perf_events` to construct accurate off-CPU and lock contention flamegraphs. 2) NUMA Node Awareness: On multi-socket NUMA hardware architectures, cross-socket memory bus hops inflate latency. Enable `-XX:+UseNUMA` to co-locate thread allocations within local memory banks. 3) Sub-Millisecond GC Engine: Deploy Generational ZGC (`-XX:+UseZGC`) to eliminate Stop-The-World latency jitter, capping max GC pauses at under 1 millisecond across multi-terabyte heaps under heavy concurrent throughput.

⚡ Modern Java 25 LTS Kodu

⚡ Modern Java 25 Implementation

```java
// Java 25: Staff+ Seviyesi - Metrik Toplama ve Sağlık Denetimi
import java.lang.management.ManagementFactory;
import java.lang.management.OperatingSystemMXBean;
import java.lang.management.ThreadMXBean;

public class StaffArchitectDiagnostics {
    public static void printSystemHealthReport() {
        OperatingSystemMXBean os = ManagementFactory.getOperatingSystemMXBean();
        ThreadMXBean threads = ManagementFactory.getThreadMXBean();

        System.out.println("================================================");
        System.out.println("🛡️ STAFF+ PRODUCTION HEALTH MONITORING REPORT");
        System.out.println("================================================");
        System.out.printf("Mevcut CPU Yükü: %.2f%%%n", os.getSystemLoadAverage() * 100);
        System.out.printf("Aktif Canlı Thread Sayısı: %d%n", threads.getThreadCount());
        System.out.printf("Zirve Thread Sayısı: %d%n", threads.getPeakThreadCount());
        System.out.printf("Toplam Başlatılan Thread: %d%n", threads.getTotalStartedThreadCount());

        long[] deadlocks = threads.findDeadlockedThreads();
        System.out.println("Deadlock Durumu: " + (deadlocks == null ? "SIFIR (Temiz)" : "🚨 " +
deadlocks.length + " Thread Kilitli!"));
        System.out.println("================================================");
    }

    void main() {
        printSystemHealthReport();
    }
}
```

⚠️ Deadlock, Race Condition & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE TEHLİKE

Canlı sistemde sorun çıkınca servisi hemen yeniden başlatmak (restart) kök nedeni sonsuza kadar yok eder! Restart atmadan önce MUTLAKA en az 3 adet thread dump ve heap dump (`jcmd \<PID> GC.heap_dump`) alın; delilleri koruyun.

⚠️ Concurrency Hazard & Interview Trap

🇬🇧 INTERVIEW TRAP & CONCURRENCY HAZARD

Blindly restarting servers during a production outage destroys forensic evidence forever! Always automate taking 3 spaced thread dumps and a memory snapshot via `jcmd \<PID> GC.heap_dump` prior to restarting instances to preserve root-cause traces.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

• Staff+ mimar ezbere çözüm üretmez; metrikleri, thread dökümlerini ve donanım mimarisini (NUMA, Cache Line, ZGC) birleştirerek kök nedeni çözer.

🎯 Key Takeaways for the Interview

• A Staff+ architect marries software abstractions with hardware topology: profilers, lock contention traces, NUMA awareness, and sub-millisecond ZGC.